



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: EMILIANO GARCIA OLVERA

N° de Cuenta: 314256009

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 23/MARZO /2026

CALIFICACIÓN: _____

Ejercicio 1: Hacer que el carro recorra la pista adecuadamente (curvas, inclinación en rampa, faro frontal ilumina la pista) desde el inicio hasta el punto extremo y ahí se detenga.

Para este ejercicio basicamente necesitamos sincronizar el movimiento del coche con la geometria de la pista. La idea es dividir el recorrido en segmentos y en cada uno ir modificando las posiciones x, y, z junto con el angulo de rotacion y (yaw) y la inclinacion x (pitch) del coche.

Lo primero fue analizar la pista segmento por segmento para identificar donde hay rectas, curvas y subidas. Con eso podemos definir intervalos de tiempo en los que el coche se comporta de cierta manera.

Variables para el movimiento del coche

Definimos las variables del coche al inicio del main, incluyendo el angulo objetivo y el angulo actual (para el suavizado).

```
float carro_x = 0.0f;
float carro_y = 0.0f;
float carro_z = 0.0f;

float carro_degrees_y_objetivo = 0.0f;
float carro_degrees_x_objetivo = 0.0f;

float carro_degrees_x = 0.0f;
float carro_degrees_y = 0.0f;

float objeto_rotacion = 0.0f;

float speed = 16.0f;
```

Segmentacion del recorrido

Antes de actualizar las posiciones, guardamos las posiciones previas para poder calcular el pitch despues. Esto porque queremos que el coche se incline automaticamente segun la pendiente real del frame.

```
float prev_y = carro_y;
float prev_z = carro_z;
float prev_x = carro_x;
```

Definimos varios segmentos basados en el timer. En cada uno el coche cambia su direccion y su velocidad en cada eje segun lo que hace la pista en ese punto.

```
// SEGMENTO 1
if (timer < 30.0f) {
    carro_degrees_y_objetivo = 0.0f;
    carro_z += dt * speed;
}
// SEGMENTO 2: curva izquierda
else if (timer < 55.0f) {
    carro_degrees_y_objetivo = 15.0f;
    carro_x += dt * speed * 0.27f;
    carro_y += dt * speed * 0.03f;
```

```

        carro_z += dt * speed * 0.95f;
    }
    // SEGMENTO 3: giro fuerte derecha + subida
    else if (timer < 80.0f) {
        carro_degrees_y_objetivo = -35.0f;
        carro_x -= dt * speed * 0.65f;
        carro_y += dt * speed * 0.22f;
        carro_z += dt * speed * 0.95f;
    }
    // SEGMENTO 4: subida casi recta, leve izquierda
    else if (timer < 120.0f) {
        carro_degrees_y_objetivo = 8.0f;
        carro_x += dt * speed * 0.12f;
        carro_y += dt * speed * 0.46f;
        carro_z += dt * speed * 1.0f;
    }
    // SEGMENTO 5: curva derecha en lo alto
    else if (timer < 138.0f) {
        carro_degrees_y_objetivo = -18.0f;
        carro_x -= dt * speed * 0.23f;
        carro_y += dt * speed * 0.19f;
        carro_z += dt * speed * 1.0f;
    }
    // SEGMENTO 6: curva derecha en lo alto
    else if (timer < 160.0f) {
        carro_degrees_y_objetivo = -18.0f;
        carro_x -= dt * speed * 0.23f;
        carro_y += dt * speed * 0.01f;
        carro_z += dt * speed * 1.0f;
    }
    // SEGMENTO 6: curva derecha en lo alto
    else if (timer < 170.0f) {
        carro_degrees_y_objetivo = -18.0f;
        carro_x -= dt * speed * 0.23f;
        carro_y -= dt * speed * 0.02f;
        carro_z += dt * speed * 1.0f;
    }
    // SEGMENTO 7: curva derecha final
    else if (timer < 181.0f) {
        carro_degrees_y_objetivo = -22.0f;
        carro_x -= dt * speed * 0.32f;
        carro_y -= dt * speed * 0.18f;
        carro_z += dt * speed * 1.0f;
    }
    // SEGMENTO 8: curva derecha final
    else if (timer < 191.0f) {
        carro_degrees_y_objetivo = -22.0f;
        carro_x -= dt * speed * 0.32f;
        carro_y -= dt * speed * 0.18f;
        carro_z += dt * speed * 1.0f;
    }
    // reset
    else {
        timer = 0.0f;

        carro_x = 0.0f;
        carro_y = 0.0f;
        carro_z = 0.0f;
    }

```

```

        carro_degrees_y_objetivo = 0.0f;
        carro_degrees_x_objetivo = 0.0f;
        carro_degrees_x = 0.0f;
        carro_degrees_y = 0.0f;

        objeto_rotacion = 0.0f;
    }

```

Como nos pide el ejercicio que la animacion se pueda repetir al presionar una tecla, al final de los segmentos hacemos un reset que reinicia todas las variables cuando el timer llega a su final (o cuando el usuario presiona R).





Inclinacion en rampa (pitch automatico)

En vez de estar adivinando el angulo de inclinacion del coche para cada segmento, calculamos el pitch automaticamente a partir de cuanto subio o bajo el coche en el frame actual. Usamos atan2 con delta_y y la distancia horizontal.

```
// Pitch automático a partir de la pendiente real del frame
float delta_y = carro_y - prev_y;
float delta_z = carro_z - prev_z;
float delta_x = carro_x - prev_x;
float horizontal = sqrt(delta_z * delta_z + delta_x * delta_x);
if (horizontal > 0.0001f) {
    carro_degrees_x_objetivo = -atan2(delta_y, horizontal) *
180.0f / 3.14159265f;
}
```

De esta forma, cuando el coche va subiendo, el morro apunta hacia arriba, y cuando baja, apunta hacia abajo. Esto le da mucho mas realismo a la animacion.

Suavizado de giros

Para que las rotaciones no se vean como teletransportes entre angulos, hacemos una interpolacion lineal hacia el angulo objetivo. Asi los giros se ven progresivos y naturales.

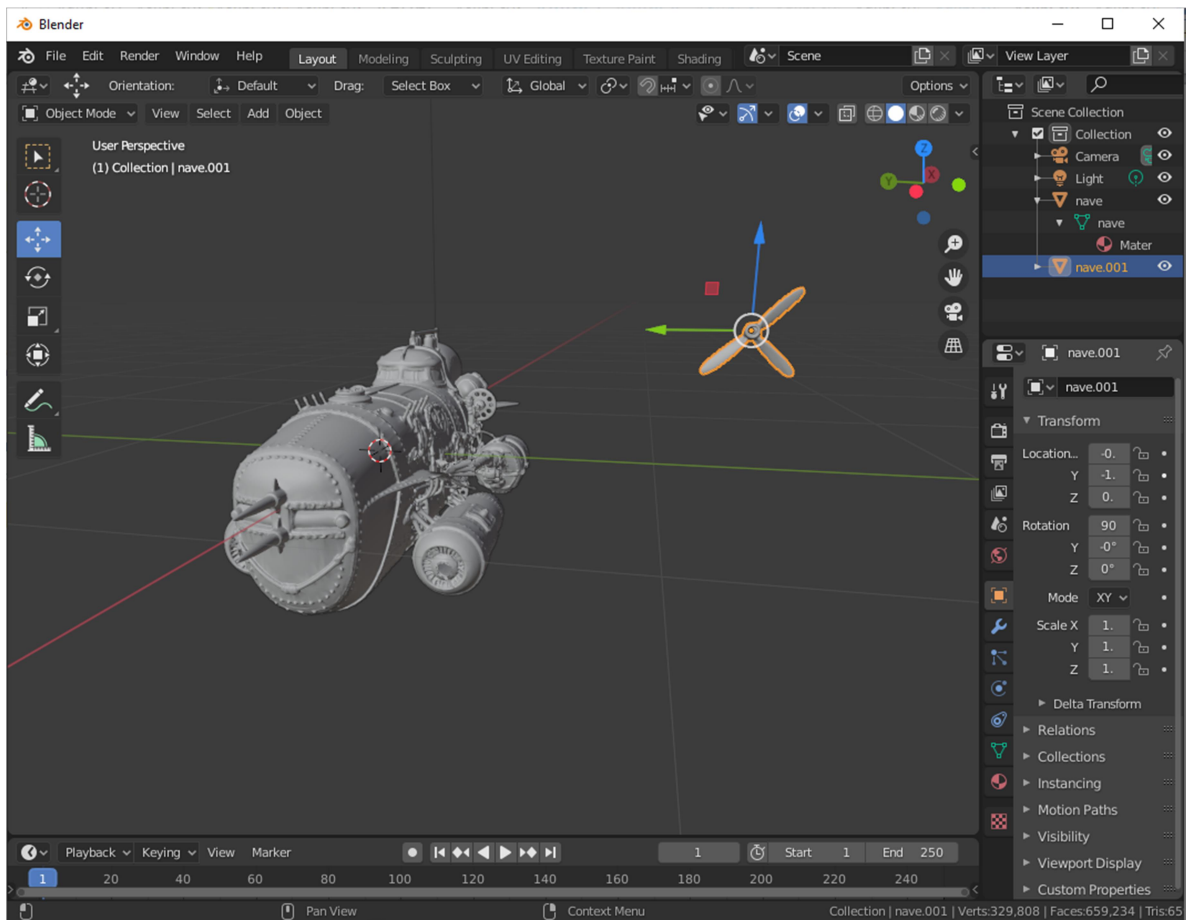
```
// Suavizado de ambas rotaciones
carro_degrees_y += (carro_degrees_y_objetivo - carro_degrees_y) *
dt * 2.0f;
carro_degrees_x += (carro_degrees_x_objetivo - carro_degrees_x) *
dt * 3.0f;
```

El factor 2.0f y 3.0f controla que tan rapido se acerca al angulo objetivo. Mas alto = giro mas brusco.

Ejercicio 2: Separar de la Nave el otra ala y las dos hélices, hacer que sobrevuele la pista adecuadamente (jerarquía, aleteo, giro de hélices, faro ilumina la pista) pero en sentido contrario, inicia en el punto extremo y aterriza junto al punto inicial. Esta animación no se puede repetir.

Separacion de partes en Blender

Lo primero fue abrir el modelo de la nave en Blender y separar las piezas en objetos independientes: la base (fuselaje), las dos alas y las dos helices. Exportamos cada uno como .obj por separado.



Despues las cargamos como modelos individuales en main.cpp.

```
Nave_M = Model();
Nave_M.LoadModel("Models/nave_base.obj");
Ala_M = Model();
Ala_M.LoadModel("Models/ala.obj");
Ala2_M = Model();
Ala2_M.LoadModel("Models/ala2.obj");
Helice_M = Model();
Helice_M.LoadModel("Models/helice.obj");
```

Variables de movimiento de la nave

Similar al coche, definimos las variables de la nave. Incluimos un bool nave_termino que nos sirve para que la animacion no se repita (a diferencia del coche). La posicion inicial de la nave es el punto extremo de la pista.

```
float nave_x = 0.0f;
float nave_y = 0.0f;
float nave_z = 0.0f;

float nave_degrees_y_objetivo = 0.0f;
float nave_degrees_x_objetivo = 0.0f;
float nave_degrees_y = 0.0f;
float nave_degrees_x = 0.0f;

float nave_timer = 0.0f;
bool nave_termino = false;

nave_x = -37.0f;
nave_y = 31.0f;
nave_z = 183.0f;
```

Recorrido inverso

La nave recorre la pista al revés que el coche. Empieza arriba (donde termina el coche) y aterriza abajo (donde empieza el coche). Los segmentos son básicamente los del coche pero invertidos en signo (x, y, z) y con ángulos también invertidos para que la nave apunte en la dirección correcta.

```
if (!nave_termino) {
    float prev_nave_x = nave_x;
    float prev_nave_y = nave_y;
    float prev_nave_z = nave_z;

    if (nave_timer < 31.0f) {
        nave_degrees_y_objetivo = -22.0f;
nave_x += dt * speed * 0.32f;
        nave_y += dt * speed * 0.08f;
        nave_z -= dt * speed * 1.05f;
    }
    // SEGMENTO 2 inverso (era SEG 5): cima en descenso
    else if (nave_timer < 71.0f) {
        nave_degrees_y_objetivo = -18.0f;
        nave_x += dt * speed * 0.23f;
        nave_y -= dt * speed * 0.13f;
        nave_z -= dt * speed * 1.0f;
    }
    // SEGMENTO 3 inverso (era SEG 4): bajada empinada
    else if (nave_timer < 106.0f) {
        nave_degrees_y_objetivo = +8.0f;
        nave_x -= dt * speed * 0.12f;
        nave_y -= dt * speed * 0.50f;
        nave_z -= dt * speed * 1.0f;
    }
    // SEGMENTO 4 inverso (era SEG 3): giro + bajada
    else if (nave_timer < 131.0f) {
        nave_degrees_y_objetivo = -35.0f;
        nave_x += dt * speed * 0.65f;
        nave_y -= dt * speed * 0.22f;
```

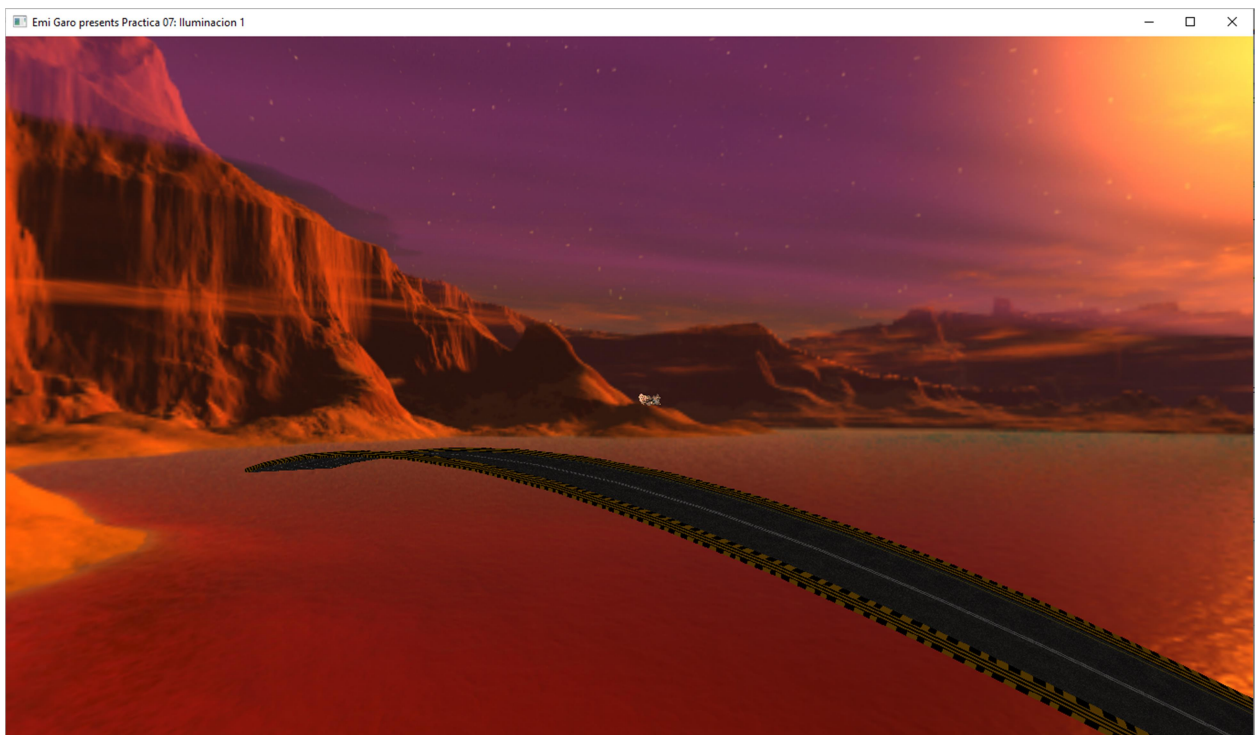
```

        nave_z -= dt * speed * 0.95f;
    }
    // SEGMENTO 5 inverso (era SEG 2): curva
    else if (nave_timer < 156.0f) {
        nave_degrees_y_objetivo = +15.0f;
        nave_x -= dt * speed * 0.27f;
        nave_y -= dt * speed * 0.09f;
        nave_z -= dt * speed * 0.95f;
    }
    // SEGMENTO 6 inverso (era SEG 1): aterrizaje en recta final
    else if (nave_timer < 180.0f) {
        nave_degrees_y_objetivo = 0.0f;
        nave_z -= dt * speed * 1.0f;

    }
    else if (nave_timer < 240.0f) {
        // leve descenso final hasta tocar piso (el -1.5 base
        + nave_y)
        nave_y -= dt * speed * 0.05f;
    }
    // Aterrizó - se detiene
    else {
        nave_termino = true;
    }
}

```

Notese que usamos el flag nave_termino para que cuando el nave_timer llega a 240 (ya aterrizo), la animacion se detenga completamente y no se repita, cumpliendo con el requisito del ejercicio.



Jerarquia de la nave

Queremos que las alas y helices sigan a la nave, pero que tambien tengan sus propias animaciones (aleteo y giro). Para esto aplicamos primero la matriz de transformacion de la nave (el padre), la guardamos en modelaux, y despues cada hija parte de esa matriz.

```
// NAVE
model = glm::mat4(1.0);
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f,
0.0f));
model = glm::translate(model, glm::vec3(nave_x, -1.5f + nave_y,
nave_z));
model = glm::rotate(model, nave_degrees_y * toRadians,
glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -nave_degrees_x * toRadians,
glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f,
0.0f));

modelaux = model; // guardamos la matriz PADRE

// Base de la nave
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
Nave_M.RenderModel();
```

Y ahora las hijas (alas y helices) parten de modelaux, asi heredan todas las transformaciones del padre y ademas aplican su propia transformacion local.

```
// Ala 1 (hereda transformación padre + aleteo propio)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.23f));
model = glm::rotate(model, ala_angulo * toRadians, glm::vec3(0.0f,
1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
Ala_M.RenderModel();

// Ala 2 (lado opuesto, aletea contrario)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.23f));
model = glm::rotate(model, -ala_angulo * toRadians,
glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
Ala2_M.RenderModel();

// Hélice 1 (giro propio)
model = modelaux;
model = glm::translate(model, glm::vec3(-0.36f, -0.29f, 0.35f));
model = glm::rotate(model, objeto_rotacion * toRadians * 100,
glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
Helice_M.RenderModel();

// Hélice 2
```

```

    model = modelaux;
    model = glm::translate(model, glm::vec3(-0.36f, -0.29f, -0.35f));
    model = glm::rotate(model, objeto_rotacion * toRadians * 100,
glm::vec3(1.0f, 0.0f, 0.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
    Helice_M.RenderModel();

```

Aleteo de alas

El aleteo lo controlamos con una bandera booleana (abriendo) y la variable ala_angulo. Cuando llega al maximo (45 grados) cambia de direccion, y cuando llega al minimo (0) vuelve a subir. Esto crea el efecto de aleteo continuo.

```

    if (abriendo == true) {
        if (ala_angulo < 45) {
            ala_angulo += dt * speed*40;
        }
        else abriendo = false;
    }
    else {
        if (ala_angulo >= 0.0) {
            ala_angulo -= dt * speed*40;
        }
        else abriendo = true;
    }

```

Giro de helices

Las helices giran usando la variable objeto_rotacion, que aumenta constantemente en cada frame. La multiplicamos por 100 para que gire rapido y se vea bien.

```

    objeto_rotacion += dt * speed * 100;

```



Faro de la nave

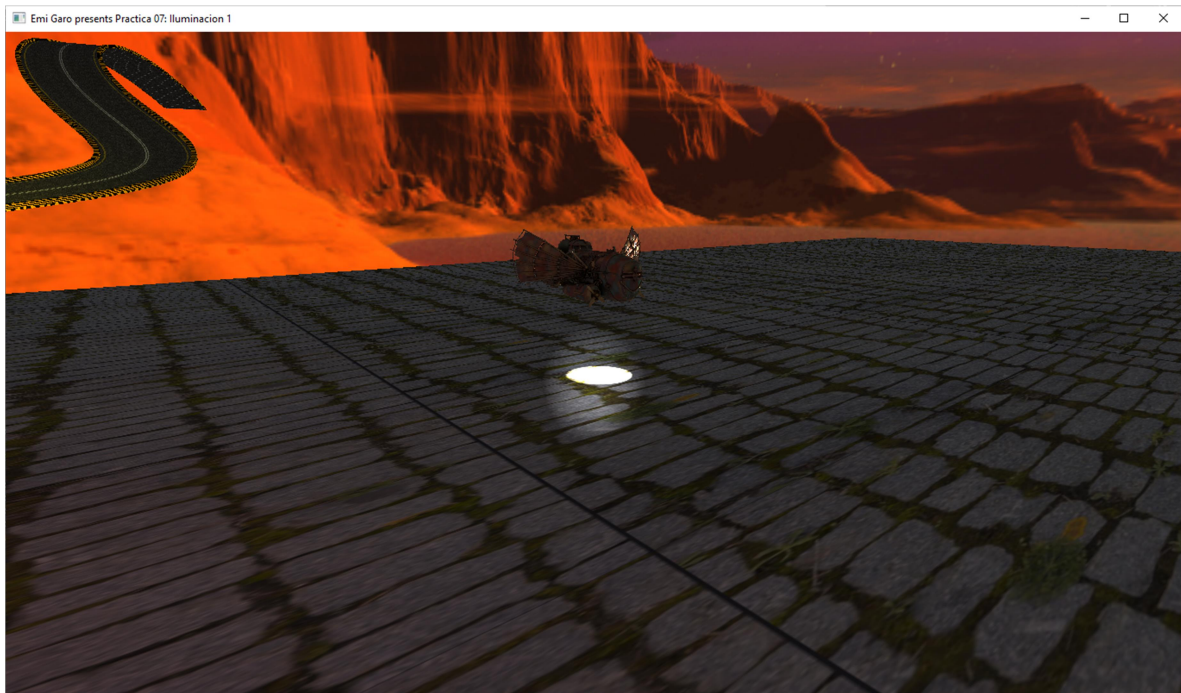
Agregamos un tercer spotlight para la nave que apunta hacia adelante (en la direccion del movimiento). Igual que el faro del coche, se actualiza con la matriz de transformacion de la nave.

```
// faro de la nave
spotLights[2] = SpotLight(1.0f, 1.0f, 0.8f, // rgb
    10.0f, 10.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 1.0f,
    1.0f, 0.09f, 0.032f, 20.0f);
spotLightCount++;
```

Y en el render de la nave, actualizamos el faro con la matriz padre.

```
// Faro de la nave (apunta al frente de la nave)
glm::mat4 faroNaveMatrix = model;
faroNaveMatrix = glm::translate(faroNaveMatrix, glm::vec3(0.0f, -
0.2f, 1.0f));
spotLights[2].SetFlash(faroNaveMatrix[3],
glm::vec3(faroNaveMatrix* glm::vec4(-1.0f, 0.0f, 0.0f, 0.0f)));
```

Notese que la direccion del faro es (0, -1, 0)



Conclusión

Durante esta practica integramos varios conceptos que habiamos visto previamente en un solo proyecto bastante completo. Por un lado, el recorrido del coche nos obligo a planear con cuidado la geometria de la pista y dividirla en segmentos con diferentes comportamientos, ademas de calcular la inclinacion del coche a partir de la pendiente real del frame, lo cual resulto mucho mas elegante que tratar de adivinar los angulos manualmente en cada segmento. El suavizado por interpolacion lineal tambien fue clave para que los giros se vieran naturales y no como teletransportes bruscos entre angulos.

Por el otro lado, con la nave repasamos el concepto de jerarquia en transformaciones. Basicamente se trata de construir una matriz padre (la nave) y que las hijas (alas y helices) partan de esa matriz para heredar todas sus transformaciones, pero tambien puedan aplicar su propia transformacion local (aleteo y giro). Esto es un concepto fundamental en graficos 3D y es exactamente lo que hacen los esqueletos de los personajes animados, donde cada hueso hereda la transformacion de su padre.

Ademas, el hecho de que la nave no se pueda repetir mientras el coche si, nos hizo pensar en como controlar el estado de las animaciones con banderas (nave_termino) para tener mas control sobre cuando una animacion se detiene de manera definitiva. Esto es util para eventos que solo deben ocurrir una vez, como aterrizajes, muertes, o cinematicas.

En general, fue una practica muy completa que junta todo lo visto hasta ahora: modelado, texturas, iluminacion (spotlights), y ahora animacion con jerarquia. Con esto ya tenemos las herramientas para construir escenas mucho mas dinamicas y complejas.

Referencias

- Apuntes de la clase de teoria y laboratorio de “Computacion Grafica e Interaccion Humano Computadora”
- De Vries, J. (n.d.). LearnOpenGL. learnopengl.com